Theory of Computation
Fall 2026

Programming assignment 3: context-free membership and dynamic programming

Deadline: Friday 30/10/2026 at 23:59

Submission is through Gradescope. This is a programming assignment. Two programming assignments together can contribute **up to 5 points** to your grade.

Late submissions are not allowed, but you can submit (and get automated feedback) as many times as you want before the deadline.

This is a group assignment. Groups can be of 1 to 3 people. Your solutions must be your own work. In particular, external help or the use of AI tools is not allowed. You should be able to explain your own solutions.

Each component of the assignment is graded on a success/failure basis, and there is no partial credit. But you can try as many times as you want until you succeed or the deadline passes. Gradescope's autograder will give you feedback about the test that your solution failed.

Overview

In this programming assignment, you will use Python to **implement** the dynamic programming algorithm that we did in class for checking if a context-free grammar, given in Chomsky Normal Form, can derive a given string. You must write a function that:

1. Receives a context-free grammar in normal form and a string of terminals, and uses dynamic programming to check if the grammar can produce the string.

It is important to note that the **proper implementation** of the required functions is crucial for this assignment, and **you must explain** how you have implemented the required constructions – **a missing or incomplete explanation can be grounds to not take your solution into account**.

The Algorithm and Input

The algorithm that you need to implement has been presented in class and can be found in the slides about the complexity class P (also in the book).

What we need as input are: the input string; the set of grammar rules, given as a dictionary; and the starting symbol for the grammar.

```
word (list): A list of terminal symbols (the input string).
rules (dict): A dictionary representing the CNF grammar.
              Keys are non-terminal symbols, values are lists of
              production rules. Each production rule is a tuple:
              - For terminal productions: (terminal_symbol,)
              - For non-terminal productions: (non_terminal_1, non_terminal_2)
```

```
            Example: {'S': [('A', 'B'), ('a',)], 'A': [('a',)], 'B':
[('b',)]}
start_symbol (str): The start symbol of the grammar.
```

Python implementations

You are required to implement the following function in your program:

```
CFG_membership(word, rules, start_symbol)
```

This function takes as input the set of terminal symbols (list); the set of rules (dict); and the starting symbol (str). It should return True or False, depending on whether word is a string that can be produced by the grammar.

Constraints and Submission Format

Do not use any Python libraries or external packages. Your functions must be given in a single file called **cfg.py**. You are provided with a template file that you can fill in. Use it as a reference to guide you. Make sure to include reasonable explanations and comments in your code to explain how your functions work. If there are no explanations of your code, it is possible that your solution will be disregarded.

Submission and feedback

You are required to submit your program in a single file called 'cfg.py' via Gradescope by the deadline. Your code will be run through an autograder, which will provide feedback on the correctness of your code. The autograder runs many tests for each implementation.

You can submit your program multiple times before the deadline, and each submission will be graded by the autograder. This will allow you to fix any errors the autograder finds and resubmit your program for re-grading.

Please make sure to review the feedback provided by the autograder carefully and address any issues that it finds before resubmitting your program.

Please note that if you have any trouble with the autograder or submitting your assignment, don't hesitate to get in touch with us for help.

Grading breakdown

The correct implementation of the function CFG_membership is worth **5 points.**